

FLAXON

A Technology-Neutral, Async-First Python Backend Framework

Computer Science Project Documentation and Working Prototype

Project proposition

Combine Flask-like ease, structured large-application development, Node.js-style asynchronous capabilities, readable debugging, optional Jinax/Jinja2 templates, and complete freedom over frontend and client technologies.

Prepared for	Computer Science Project / Framework Development
Framework name	Flaxon
Primary language	Python 3.11+
Prototype version	0.1.0 Alpha
Prepared by	Aldane Hutchinson

July 2026

2. Abstract and Executive Summary

Flaxon is a proposed open-source Python backend framework for APIs, server-rendered websites, real-time applications, mobile backends, and structured enterprise systems. The project addresses a recurring problem in Python web development: simple frameworks are easy to start but require significant manual architecture as applications grow, while larger frameworks provide structure by controlling more technical decisions. Developers also frequently select JavaScript runtimes for chat, notification, streaming, and high-concurrency workloads even when their teams prefer Python.

The framework therefore adopts an async-first ASGI architecture, Flask-style route decorators, optional modular structure, middleware, request validation, WebSockets, lifecycle hooks, testing utilities, a command-line interface, and a debugger designed to explain failures in direct language. Flaxon does not attempt to make Python a compiled language or guarantee that it will outperform every Node.js, Java, Go, or Rust service. Its objective is to reduce avoidable overhead, use asynchronous I/O correctly, and make development speed and operational clarity first-class requirements.

Jinax is the optional server-side template integration. Instead of inventing an unsafe or incomplete template language for the first release, the prototype wraps Jinja2—the same template engine commonly associated with Flask—while giving Flaxon-specific configuration, async rendering, response helpers, filters, and diagnostics. Applications using React, Vue, Angular, Kotlin, Java, Flutter, Capacitor, C#, Rust, or plain API clients never need to install or load Jinax.

- Deliverable 1: a thirty-page computer science project document covering requirements, design, implementation, testing, security, deployment, risks, and roadmap.
- Deliverable 2: a runnable source prototype with ASGI HTTP routing, typed parameters, JSON responses, middleware, schemas, WebSockets, Jinax, testing tools, CLI commands, Docker files, and examples.
- Prototype verification: eight automated tests passed, CLI route inspection succeeded, and live Uvicorn smoke tests returned successful health and validated user responses.

Keywords

Python framework, ASGI, backend development, WebSocket, Android backend, Jinja2, template engine, readable debugger, middleware, validation, technology neutrality.

3. Table of Contents

No.	Section	Page
1	Cover Page	1
2	Abstract and Executive Summary	2
3	Table of Contents	3
4	Introduction and Project Background	4
5	Problem Statement and Opportunity	5
6	Aim, Objectives, and Success Criteria	6
7	Scope, Assumptions, and Limitations	7
8	Comparative Analysis	8
9	Functional Requirements	9
10	Non-Functional Requirements	10
11	System Architecture	11
12	ASGI and Request Lifecycle	12
13	Application, Configuration, and Lifecycle	13
14	Routing, Requests, and Responses	14
15	Middleware and Security Foundation	15
16	Validation and Readable Debugger	16
17	WebSocket and Real-Time Design	17
18	Jinax Template Integration	18
19	Database and Plugin Independence	19
20	CLI and Developer Experience	20
21	Android and Mobile Backend Strategy	21
22	Production Repository Structure	22
23	Working Prototype Implementation	23
24	Example Applications and Usage	24
25	Testing and Verification Results	25
26	Performance and Scalability Plan	26
27	Security, Privacy, and Reliability	27
28	Deployment and DevOps	28
29	Roadmap, Feasibility, Risks, and Conclusion	29
30	References and Execution Checklist	30

Document navigation note: each major section begins on the listed page. The prototype ZIP contains the full source files referenced throughout this report.

4. Introduction and Project Background

Web and mobile systems increasingly require one backend to serve many clients. A single product may have a React website, an Android application, an administrative dashboard, scheduled jobs, external integrations, and a real-time chat channel. The backend must expose stable HTTP and WebSocket interfaces while remaining understandable to developers and operators. Flaxon is designed around this multi-client reality rather than around one preferred user-interface stack.

Python is attractive because of its readable syntax, broad ecosystem, strong educational use, automation capabilities, data tooling, and developer productivity. However, backend teams may face three practical choices: begin with a minimal framework and assemble architecture manually; adopt a batteries-included framework with strong conventions; or switch to a different runtime when concurrency and real-time communication become central. Flaxon explores a fourth option: retain Python while combining a small entry experience with optional production structure and async-first networking.

The project is not a clone of Flask, Spring Boot, or Node.js. Flask influences the route ergonomics and readable development errors. Spring Boot influences modular services, configuration, lifecycle management, and a path toward large applications. Node.js influences the focus on non-blocking I/O, WebSockets, event-oriented systems, and developer-friendly package workflows. The actual technical foundation is ASGI, which defines application interfaces for HTTP, WebSocket, and lifespan events [1][2].

- Small applications should start in one file without generators or mandatory architecture.
- Large applications should be able to introduce routers, services, repositories, middleware, plugins, workers, and observability without migrating frameworks.
- HTML rendering is optional; JSON APIs are a first-class default.
- Framework APIs should be explicit enough to debug and profile without hidden magic.
- Production claims must be supported by tests, measurements, security review, and documented limitations.

Core principle

Simple applications remain simple; large applications gain structure without losing technology choice.

5. Problem Statement and Opportunity

The main problem is fragmentation between simplicity, structure, concurrency, and freedom. A developer can create a Flask application quickly, but a growing system still needs decisions for configuration, modular routing, validation, WebSockets, background work, security, testing, observability, and deployment. A framework such as Django provides many integrated capabilities, but its architecture may be more opinionated than an API-first or mobile-first team requires. Java and Spring Boot are strong for structured enterprise services but typically involve a different language ecosystem and more ceremony. Node.js supports event-driven applications well, yet Python teams may not want to maintain a second backend language.

A second problem is debugging at scale. A raw traceback identifies the call stack but does not always explain the request, matched route, validation context, middleware chain, request identifier, or likely correction. During development, failures should be direct and educational. In production, the client should receive a safe error code and request identifier while complete details are stored securely for operators.

A third problem affects mobile development. Android developers commonly use Kotlin or Java for the interface, but still need authentication, synchronization, uploads, notifications, chat, permissions, and analytics services. Python can serve this backend role effectively when the network layer and shared state are designed correctly. Flaxon targets this area without pretending that Python should replace native Android user-interface tools.

Observed difficulty	Flaxon response
Minimal frameworks need assembly	Provide optional built-ins and documented extension points.
Large frameworks can constrain choices	Keep databases, frontends, ORMs, and clients replaceable.
Real-time features push teams elsewhere	Make ASGI and WebSockets part of the core.
Tracebacks are difficult for beginners	Produce structured, redacted, request-aware errors.
Mobile teams need backend services	Offer stable JSON/WebSocket patterns and mobile examples.

Research question

Can a Python framework preserve Flask-like approachability while providing an async core, optional large-scale structure, and clearer diagnostics without controlling the developer's frontend or database?

6. Aim, Objectives, and Success Criteria

Aim: design and prototype a Python backend framework that is easy to code, async-first, technology-neutral, suitable for APIs and real-time applications, and easier to debug as systems become larger.

Specific objectives

1. Create an ASGI application object that handles HTTP, WebSocket, startup, and shutdown events.
2. Provide Flask-style decorators, route groups, typed path parameters, named routes, and URL generation.
3. Return JSON automatically while supporting text, HTML, redirects, and streaming response classes.
4. Implement middleware for request identifiers, CORS, security headers, and rate limiting.
5. Resolve validated schema objects directly into route parameters and return consistent 422 errors.
6. Support WebSocket connections, JSON messages, rooms, and a replaceable broadcasting layer.
7. Provide Jinax as an optional Jinja2-powered integration with autoescaping and async rendering.
8. Create a debugger that separates development detail from production-safe error responses.
9. Deliver a CLI for running, inspecting, checking, and generating Flaxon projects.
10. Package the framework using a modern pyproject.toml source layout and verify it with tests.

Prototype success criteria

Criterion	Required evidence
Runnable	Uvicorn starts the application and serves real requests.
Correct routing	Static, typed, prefixed, missing, and method cases behave consistently.
Validation	Valid inputs resolve; invalid inputs identify fields.
Template choice	API examples work without Jinax; HTML example works with Jinax.
Debug safety	Sensitive names are redacted and production errors hide traces.
Developer usability	README, CLI, tests, Docker, and examples are included.
Extensibility	Middleware, routers, template engine, and services can be replaced.

7. Scope, Assumptions, and Limitations

The project scope covers a credible framework foundation and a working alpha prototype. It does not claim to be a complete replacement for mature frameworks. The prototype is intended for learning, evaluation, experimentation, and continued engineering.

Included scope

- ASGI 3 callable; HTTP, WebSocket, and lifespan dispatch.
- Decorator routing; Flask-style and brace-style typed parameters; route groups and URL generation.
- Request parsing for headers, query values, cookies, body, text, and JSON.
- Response classes for JSON, text, HTML, redirect, bytes, and streaming content.
- Middleware stack with CORS, request ID, security headers, and a single-process rate limiter.
- Declarative validation schemas with string, email, integer, float, boolean, and choice fields.
- In-memory WebSocket rooms and optional Jinax/Jinja2 template rendering.
- Development/production error separation, redaction, test client, CLI, Docker, CI, and examples.

Excluded from version 0.1

- A proprietary production ASGI network server; Flaxon runs on a dedicated ASGI server.
- A mandatory ORM or migration system. Database integrations should be plugins.
- Distributed queues, Redis broadcasting, OpenAPI generation, sessions, OAuth, and admin UI as finished features.
- Claims of internet-scale performance without independent benchmarks and operational history.
- Automatic support for every protocol edge case. ASGI conformance testing remains a pre-1.0 requirement.

Assumptions: developers use Python 3.11 or later; production deployments use HTTPS through a suitable platform or reverse proxy; shared state is moved out of process when multiple workers are used; and blocking libraries are isolated or replaced with async-compatible clients.

Honest limitation

Async Python improves concurrent I/O, but it does not automatically speed up CPU-heavy work. Image processing, machine learning inference, compression, or complex calculations may require worker processes, native extensions, or separate services.

8. Comparative Analysis

Characteristic	Flask	Spring Boot	Node.js ecosystem	Flaxon goal
Primary language	Python	Java/Kotlin	JavaScript/TypeScript	Python
Starting experience	Very small	Structured project	Varies by framework	Very small
Async/realtime core	Not original core focus	Supported	Strong ecosystem focus	ASGI-first
Large-app structure	Developer-defined	Strong conventions	Framework-dependent	Optional conventions
Template approach	Jinja2	Multiple engines	Many choices	Optional Jinax/Jinja2
Database choice	Open	Open through ecosystem	Open	Open through adapters/plugins
Client technology	Any	Any	Any	Any
Debug objective	Readable local errors	Enterprise tooling	Varies	Request-aware explanations

The comparison is conceptual rather than a benchmark. Flaxon borrows useful developer-experience patterns while retaining its own boundaries. It should not recreate Spring's complete dependency-injection and enterprise ecosystem during the first releases. It should not copy the entire Node.js package culture or claim event-loop equivalence under every workload. It should not replace Jinja2 with a new parser unless a future design has a measurable advantage and receives security review.

The strongest differentiation is the combination of technology neutrality and understandable growth. A student can begin with one Python file. A team can later split routes, schemas, services, repositories, event handlers, and plugins. The same backend may serve an HTML page, React SPA, Android application, and WebSocket client. The framework owns transport and developer ergonomics; it does not own the product's user-interface language.

Positioning statement: Flaxon is for Python developers who want a small entry point, modern asynchronous networking, clear errors, mobile/API friendliness, and an upgrade path to structured systems without adopting a mandatory full-stack architecture.

Design decision

Jinax is an integration name, not a new template syntax in version 0.1. It uses Jinja2 because template inheritance, autoescaping, inclusion, and async rendering are already established capabilities [3][4].

9. Functional Requirements

ID	Requirement	Priority
FR-01	Create an application with a name, configuration, and debug mode.	Must
FR-02	Register HTTP routes using decorators and routers.	Must
FR-03	Convert typed path values and report missing routes/methods.	Must
FR-04	Read headers, query values, cookies, bytes, text, and JSON.	Must
FR-05	Generate JSON, text, HTML, redirects, and streaming responses.	Must
FR-06	Run middleware before and after framework dispatch.	Must
FR-07	Validate request bodies using schemas and field definitions.	Must
FR-08	Accept WebSocket connections and exchange text/JSON messages.	Must
FR-09	Execute startup and shutdown callbacks.	Must
FR-10	Render HTML through optional Jinax/Jinja2 integration.	Should
FR-11	List routes and run environment checks through a CLI.	Should
FR-12	Provide synchronous and asynchronous testing clients.	Should
FR-13	Allow replaceable database, cache, queue, and broadcaster plugins.	Should
FR-14	Generate OpenAPI documentation in a later milestone.	Could
FR-15	Provide distributed tasks and scheduling in a later milestone.	Could

Primary use cases

- Build a JSON API for a React or Vue application.
- Provide authentication, uploads, notifications, and synchronization for Android/Kotlin.
- Create a real-time chat or presence service with WebSockets.
- Render a small business website with Jinax while using the same service layer as the API.
- Develop a modular marketplace backend with routers, schemas, services, and repositories.
- Run health checks and inspect routes before deployment.

The “framework does not care” requirement is implemented as interface freedom, not the absence of standards. Clients must still communicate using supported protocols and agreed schemas. Flaxon should care about correctness, security, observability, and compatibility while remaining neutral about React versus Vue, PostgreSQL versus MongoDB, or Kotlin versus Flutter.

10. Non-Functional Requirements

Area	Requirement and measurable direction
Usability	A first API should require fewer than ten lines; error messages should identify route, request ID, problem, and safe next action.
Performance	Minimize allocations and middleware overhead; benchmark routing, JSON, templates, and WebSockets before release claims.
Scalability	Support stateless multi-worker deployment; externalize sessions, cache, jobs, and WebSocket broadcasting.
Security	Autoescape HTML; redact secrets; provide secure headers; reject malformed input; document HTTPS and proxy requirements.
Reliability	Use lifecycle cleanup, consistent exceptions, health endpoints, timeouts, and graceful shutdown guidance.
Maintainability	Use a src layout, type hints, small modules, tests, semantic versioning, change log, and compatibility policy.
Portability	Run on Windows, Linux, containers, and common cloud platforms where Python and an ASGI server are supported.
Interoperability	Expose standards-based HTTP, JSON, WebSocket, and ASGI behavior; avoid private client protocols.
Observability	Attach request IDs; provide structured logs and future OpenTelemetry/Sentry plugins.
Accessibility	Generated documentation and default error pages should use semantic HTML and readable contrast.

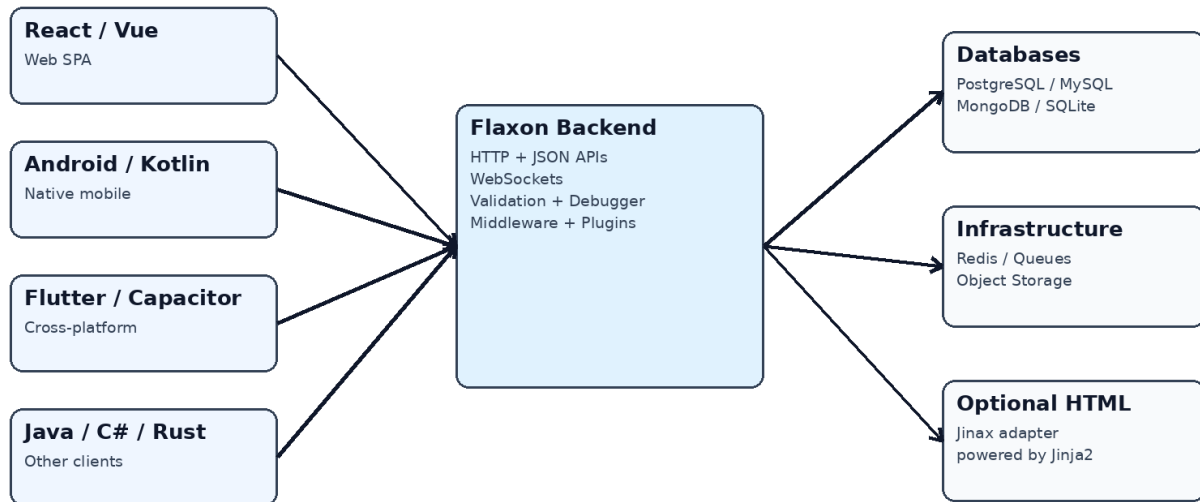
Quality gates for a stable release should include protocol conformance, unit and integration coverage, static analysis, security scanning, dependency review, memory profiling, multi-version Python CI, load tests, and documented upgrade paths. Passing a small local test suite is necessary for the prototype but not sufficient for a production guarantee.

Performance definition

“Fast” means measured low overhead for its intended workloads and efficient concurrency for I/O-bound applications—not an unqualified claim that Python is always faster than Node.js, Java, Go, or Rust.

11. System Architecture

Flaxon Technology-Neutral Architecture



Flaxon sits between clients and application services. Any client capable of HTTP or WebSocket communication can use it. The core handles protocol dispatch, routing, request/response objects, middleware, validation, errors, lifecycle events, and extension registration. Business logic remains in the developer's application services rather than in controllers or templates.

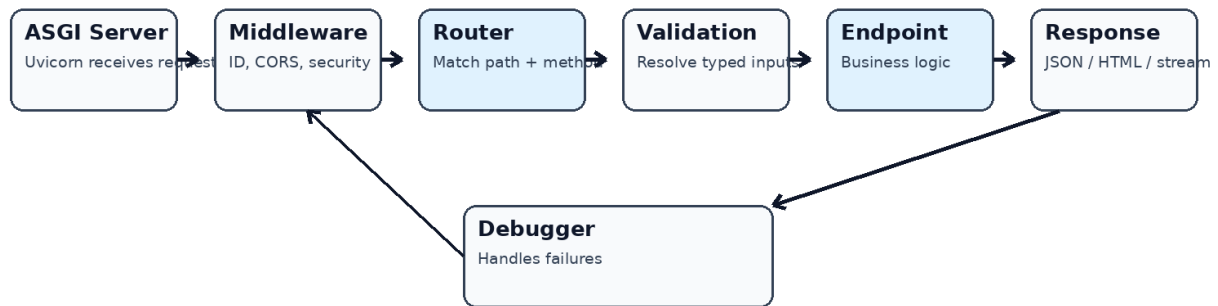
The framework uses layered boundaries:

- Protocol layer: ASGI scope, receive, and send messages.
- Framework layer: routing, middleware, validation, WebSockets, Jinax, debugging, testing, and CLI.
- Application layer: controllers/routes, services, policies, schemas, and events.
- Infrastructure layer: database, Redis, queues, storage, email, push notifications, and external APIs.
- Client layer: web, Android, desktop, command-line, or third-party integrations.

Dependency direction should point inward: application business rules should not depend directly on HTTP details when avoidable. This makes the same service usable from a REST route, WebSocket event, scheduled task, or test.

12. ASGI and Request Lifecycle

Flaxon HTTP Request Lifecycle



ASGI applications receive a connection scope and two asynchronous callables: receive and send. HTTP requests, WebSocket connections, and lifecycle events use defined message types [1][2]. Flaxon implements one public application callable and dispatches by scope type.

1. The ASGI server creates a scope and forwards protocol messages.
2. Middleware adds cross-cutting behavior such as request IDs, CORS, security headers, limits, logging, and recovery.
3. The router matches path and method, converts parameters, and returns the endpoint.
4. The invoker resolves Request objects, path parameters, and Schema annotations.
5. The endpoint performs application logic and returns a response-compatible value.
6. Flaxon converts dictionaries/lists to JSON and sends ASGI response messages.
7. Exceptions are mapped to safe HTTP errors or debug information according to environment.

Lifespan support is important because database pools, caches, and other resources should be created after process startup and closed during graceful shutdown. The prototype executes startup handlers in registration order and shutdown handlers in reverse order.

Concurrency rule

Async endpoints must avoid blocking calls. A synchronous database driver or long CPU task can block the event loop even when the route is declared with `async def`.

13. Application, Configuration, and Lifecycle

The Flaxon class is the central registry and ASGI entry point. It owns configuration, routes, middleware definitions, lifecycle callbacks, application state, Jinax configuration, WebSocket room management, and the debugger. The constructor remains small:

```
from flaxon import Flaxon

app = Flaxon(
    "marketplace-api",
    debug=True,
    config={"ENV": "development"},
)
```

Configuration values can come from defaults, a dictionary, or environment variables prefixed with `FLAXON_`. The prototype coerces common booleans, integers, and comma-separated lists. A production version should add validated settings classes, `.env` integration as an optional package, secret-manager adapters, environment-specific files, and warnings for insecure combinations.

```
@app.on_startup
async def connect_resources():
    app.state.database = await create_database_pool()

@app.on_shutdown
async def close_resources():
    await app.state.database.close()
```

Application state is intentionally explicit. It is suitable for long-lived resources such as pools and service registries, not per-request user data. Per-request state belongs on the Request object. Multi-worker applications must not treat process memory as shared durable state.

Configuration area	Production recommendation
DEBUG	False; never expose tracebacks publicly.
SECRET_KEY	Random, long, rotated, and stored outside source control.
ALLOWED_HOSTS	Explicit domains rather than wildcard acceptance.
MAX_BODY_SIZE	Set according to upload requirements and proxy limits.
ENV	Use development, testing, staging, and production profiles.
Logging	Structured output with request and trace identifiers.

14. Routing, Requests, and Responses

Routes should be easy to read while supporting growth. The prototype accepts Flask-style parameters such as `<int:user_id>` and brace-style parameters such as `{user_id:int}`. Built-in converters include `str`, `int`, `float`, `path`, and `UUID`. Route names support reverse URL generation.

```
from flaxon import Router

api = Router(prefix="/api/v1")

@api.get("/users/<int:user_id>", name="users.detail")
async def user_detail(user_id: int):
    return {"id": user_id}

app.include_router(api)
url = app.url_for("users.detail", user_id=42)
```

The `Request` object exposes `method`, `path`, `URL`, `headers`, `cookies`, `query values`, `path parameters`, `client information`, `user`, `state`, and `async body methods`. Body bytes are cached after the first read so validation and endpoint code can safely refer to the same request content.

```
@app.post("/api/products")
async def create_product(request):
    payload = await request.json()
    return {"name": payload["name"], "price": payload["price"]}
```

Return conversion is intentionally convenient: dictionaries, lists, tuples, and dataclasses become JSON; strings become text; bytes become an octet-stream response; `None` becomes 204; and explicit `Response` objects keep their chosen status, headers, and media type.

Response type	Typical use
<code>JSONResponse</code>	APIs and mobile clients
<code>HTMLResponse</code>	Jinax or manually generated HTML
<code>TextResponse</code>	Health, robots, or simple diagnostics
<code>RedirectResponse</code>	Navigation and authentication flows
<code>StreamingResponse</code>	Large downloads, events, generated output

15. Middleware and Security Foundation

Middleware wraps the ASGI application and is appropriate for behavior that applies across routes. The prototype includes request identifiers, security headers, CORS, and a basic in-memory rate limiter. Future middleware should cover trusted hosts, proxy headers, compression, body limits, timeouts, sessions, access logs, metrics, and exception recovery.

```
from flaxon.middleware import CORSMiddleware
from flaxon.security import RateLimitMiddleware

app.add_middleware(
    CORSMiddleware,
    allowed_origins=["https://app.example.com"],
    allow_credentials=True,
)

app.add_middleware(RateLimitMiddleware, requests=120, window_seconds=60)
```

CORS is a browser security policy, not an authentication system. Allowed origins should be explicit when credentials are used. The default security header middleware sends nosniff, frame denial, referrer policy, and a restrictive permissions policy. A production security plugin should allow application-specific Content Security Policy, HSTS at the TLS boundary, secure cookie configuration, and nonce generation.

The supplied rate limiter is deliberately labeled single-process. Multiple workers would each have separate counters, so production limits require a shared store such as Redis or enforcement at an API gateway. This distinction is important: a prototype must not present local process state as distributed protection.

Middleware order concern	Reason
Proxy/host validation early	Establish trustworthy client and host information.
Request ID before logging	All later logs and errors share one identifier.
CORS around responses	Errors also need correct browser headers.
Authentication before authorization	Permissions require a verified identity.
Compression late	Compress the completed response, not intermediate data.
Recovery outermost	Catch failures from routes and inner middleware.

16. Validation and Readable Debugger

Flaxon schemas turn request contracts into code. A Schema metaclass collects Field declarations. When an endpoint parameter is annotated with a Schema subclass, the framework reads JSON, validates each field, and injects the resulting object. Invalid input returns status 422 with field-specific messages.

```
from flaxon.validation import Schema, fields

class CreateUser(Schema):
    name = fields.String(required=True, min_length=2)
    email = fields.Email(required=True)
    age = fields.Integer(minimum=13, maximum=120)

@app.post("/users")
async def create_user(data: CreateUser):
    return {"user": data.to_dict()}
```

The debugger maps expected HTTPException instances to stable codes and safe details. Unexpected exceptions behave differently by environment. In production, the client receives FX-SRV-500 and a request ID. In development, JSON clients receive exception type, message, path, parameters, query data, and traceback. Browser requests can receive a readable HTML page. Keys containing password, secret, token, authorization, cookie, API key, private key, or credit-card terms are redacted.

Example production response

```
{"success":false,"error":{"code":"FX-SRV-500","message":"The request could not be completed.","request_id":"..."}}
```

A future large-scale debugger should add middleware timelines, database query snapshots, task context, WebSocket events, source excerpts, suggested fixes, error grouping, and integrations with Sentry or OpenTelemetry. It must apply strict access control and data-retention rules because debug snapshots may contain personal or confidential information.

17. WebSocket and Real-Time Design

WebSockets provide a persistent, full-duplex channel suitable for chat, presence, notifications, live dashboards, collaborative editing, game events, and mobile synchronization. The prototype exposes `accept`, `receive_text`, `receive_json`, `send_text`, `send_json`, `close`, `iter_json`, `join`, `leave`, and room broadcast helpers.

```
@app.websocket("/ws/chat/<room_id>")
async def chat(socket, room_id: str):
    await socket.accept()
    await socket.join(room_id)
    try:
        async for message in socket.iter_json():
            await socket.broadcast_json(room_id, {
                "event": "chat.message",
                "data": message,
            })
    finally:
        await socket.leave(room_id)
```

The included `WebSocketManager` stores room membership in process memory. This is correct for local development and a single worker. It is not sufficient for horizontal scaling because a socket connected to worker one is invisible to worker two. The production design therefore defines a replaceable broadcaster interface. A Redis pub/sub or streams adapter can distribute room events across workers while each worker maintains its own active socket objects.

- Authenticate during the handshake or first message and close unauthorized connections.
- Validate every message; do not trust client event names or payload sizes.
- Use heartbeat/ping logic and idle timeouts to remove dead connections.
- Apply per-user and per-room rate limits to avoid message floods.
- Persist important chat messages before broadcasting when durability is required.
- Use backpressure and queue limits so slow clients do not consume unbounded memory.

Instagram-like systems

Flaxon can support the API and real-time portions of a social application, but media processing, feed ranking, search, storage, moderation, and global delivery require additional specialized services.

18. Jinax Template Integration

Jinax is Flaxon's optional server-rendered HTML integration. Version 0.1 uses Jinja2 rather than creating a custom parser. This provides mature template inheritance, includes, filters, macros, autoescaping, and async rendering [3][4]. The "Jinax" name represents Flaxon-specific setup and response helpers while preserving standard Jinja syntax and ecosystem knowledge.

```
from flaxon.jinax import Jinax

app.use_templates(Jinax(
    "templates",
    auto_reload=app.debug,
    strict_undefined=True,
))

@app.get("/")
async def home(request):
    return await request.render("home.html", {
        "title": "Flaxon Store",
        "products": await product_service.list(),
    })

{% extends "base.html" %}
{% block content %}
<h1>{{ title }}</h1>
{% for product in products %}
<article>{{ product.name }} — {{ product.price | currency("JMD") }}</article>
{% else %}
<p>No products are available.</p>
{% endfor %}
{% endblock %}
```

Autoescaping is selected for HTML and XML templates. Strict undefined values make template mistakes fail clearly during development. Jinax adds a currency filter and APIs for custom globals and filters. Later releases can add Flaxon-aware URL generation, CSRF helpers, component conventions, template diagnostics, fragment caching, and static-asset manifests.

Technology freedom

A JSON API project should not install Jinax. React, Vue, Android, Flutter, Java, Rust, and C# clients consume Flaxon through HTTP/WebSockets and choose their own interface technologies.

19. Database and Plugin Independence

Flaxon should not force one ORM, database, cache, queue, storage service, or authentication package. Instead, it should define lifecycle and extension contracts. Developers may use SQLAlchemy, SQLModel, asyncpg, psycopg, Tortoise ORM, PyMongo/Motor, SQLite, Redis, or custom libraries. The framework helps initialize resources, inject services, manage transactions, expose health checks, and translate known infrastructure errors without owning the application's data model.

```
@app.on_startup
async def startup():
    app.state.db = await asyncpg.create_pool(DATABASE_URL)

@app.on_shutdown
async def shutdown():
    await app.state.db.close()

@app.get("/users/<int:user_id>")
async def get_user(user_id: int):
    row = await app.state.db.fetchrow(
        "SELECT id, name FROM users WHERE id=$1", user_id
    )
    return dict(row) if row else {"error": "not found"}
```

A formal plugin protocol should include metadata, configuration registration, startup, shutdown, route registration, middleware registration, health checks, CLI commands, and compatibility ranges. Plugins must not silently modify unrelated application behavior. The framework should expose clear hook points and produce warnings for incompatible versions.

Proposed plugin	Responsibility
flaxon-sqlalchemy	Async engine lifecycle, sessions, repositories, migrations.
flaxon-redis	Cache, distributed rate limiting, WebSocket broadcast.
flaxon-auth	JWT, sessions, OAuth, roles, permissions.
flaxon-tasks	Redis/database queues, workers, retries, schedules.
flaxon-observability	OpenTelemetry, metrics, traces, structured logs.
flaxon-storage	Local and object-storage upload adapters.

20. CLI and Developer Experience

The command-line interface shortens common workflows without hiding the generated code. The prototype uses Python's `argparse` module and exposes four commands: `run`, `routes`, `doctor`, and `new`. A later CLI can add `shell`, `test`, `build`, `inspect`, `generate`, `migrate`, `worker`, `schedule`, and `plugin` commands.

```
flaxon new marketplace
flaxon run app:app --reload
flaxon routes app:app
flaxon doctor app:app
```

The `routes` command prints method, path, and route name, including WebSockets. `Doctor` imports the application, counts routes, identifies duplicate registrations, warns about weak or missing secrets, and detects debug mode in production. These commands support the project goal of making large applications easier to understand.

Command	Expected behavior
<code>new</code>	Create <code>app.py</code> , <code>pyproject.toml</code> , <code>README</code> , and <code>.gitignore</code> .
<code>run</code>	Start Uvicorn using an import string; support host, port, reload, and workers.
<code>routes</code>	Display all HTTP and WebSocket endpoints.
<code>doctor</code>	Check imports, counts, duplicates, environment, and security warnings.
<code>generate (planned)</code>	Create controllers, schemas, services, middleware, tasks, and plugins.
<code>inspect (planned)</code>	Show middleware order, dependencies, configuration sources, and health.

Developer experience also includes type hints, meaningful names, stable error codes, minimal boilerplate, fast test execution, high-quality examples, and a migration policy. The official documentation should have two paths: a beginner quick start and an architecture reference for experienced teams.

Avoid magic

Generated structures are recommendations, not requirements. A one-file application remains valid, and generated files should be ordinary Python that developers can read and edit.

21. Android and Mobile Backend Strategy

Flaxon targets Android on the backend side. Kotlin or Java remains the recommended native interface technology, while React Native, Flutter, Ionic, or Capacitor may provide cross-platform interfaces. Flaxon exposes versioned APIs and WebSocket services that mobile clients call over the network.

```
class DeviceRegistration(Schema):
    device_id = fields.String(required=True, min_length=8)
    platform = fields.Choice(["android"], required=True)
    notification_token = fields.String(required=True, min_length=16)

@app.post("/api/v1/devices")
async def register_device(data: DeviceRegistration):
    return {"success": True, "device": data.to_dict()}
```

Mobile-oriented framework guidance should cover:

- Access and refresh tokens with secure rotation and server-side revocation.
- API versioning, backwards compatibility, and deprecation periods.
- Idempotency keys for payment, upload, and synchronization requests.
- Chunked or direct-to-object-storage media uploads.
- Push notification adapters such as Firebase Cloud Messaging.
- Offline-first synchronization using timestamps, revisions, conflict policy, and delta endpoints.
- Device registration, session visibility, remote logout, and suspicious-device alerts.
- Bandwidth-aware pagination, compression, caching, and image variants.
- WebSocket reconnection, missed-event recovery, and background limitations on mobile operating systems.

An example architecture may use Kotlin + Retrofit for HTTP, a WebSocket client for chat, Flaxon for API logic, PostgreSQL for durable data, Redis for ephemeral state, object storage for media, and a notification service for delivery. This attracts Android developers to Python backend development without misrepresenting Python as an Android UI runtime.

22. Production Repository Structure

The full vision uses a source layout and separates stable framework concerns. The supplied prototype implements the boldest early modules and leaves expansion points for later packages.

```
flaxon-framework/
├── pyproject.toml, README.md, LICENSE
├── Dockerfile, docker-compose.yml
├── .github/workflows/tests.yml
├── src/flaxon/
│   ├── application/      # app, config, lifecycle, state
│   ├── routing/          # router, route, converters
│   ├── http/             # request, response, status
│   ├── middleware/       # base, CORS, request ID, headers
│   ├── validation/       # schema, fields, errors
│   ├── websocket/        # connection, rooms/manager
│   ├── jinx/             # optional Jinja2 integration
│   ├── debugging/        # debugger and redaction
│   ├── security/         # rate limiting foundation
│   ├── testing/          # async and sync test clients
│   ├── cli/              # run, routes, doctor, new
│   ├── server/           # Uvicorn runner
│   └── utils/
├── examples/
│   ├── hello_api/
│   ├── jinx_site/
│   ├── react_backend/
│   └── android_backend/
├── tests/
│   ├── test_core.py
│   ├── test_validation.py
│   ├── test_errors.py
│   └── test_jinx.py
```

Future production modules can add database adapters, transactions, sessions, authentication, authorization, caching, task queues, events, OpenAPI, file storage, mail, health checks, metrics, plugin discovery, and richer debugging. Keeping these as modules or optional distributions prevents the core installation from becoming unnecessarily heavy.

The `pyproject.toml` file uses the standard project metadata table, optional dependency groups, a console-script entry point, and a `src` package layout. Modern Python packaging guidance treats `pyproject.toml` as the central configuration for build tools and project metadata [5].

23. Working Prototype Implementation

The included ZIP is not only a design skeleton. It contains executable framework code and examples. The central Flaxon class is an ASGI callable that builds the middleware stack, handles lifecycle messages, constructs Request and WebSocket objects, resolves routes, injects schemas, converts return values, and delegates exceptions to the debugger.

```
async def __call__(self, scope, receive, send):
    if self._middleware_stack is None:
        self._middleware_stack = self._build_middleware_stack()
    await self._middleware_stack(scope, receive, send)

async def _handle_http(self, scope, receive, send):
    request = Request(scope, receive, self)
    try:
        match = self.router.match(request.path, request.method)
        value = await self._invoke_http(
            match.route.endpoint, request, match.params
        )
        response = Response.from_value(value)
    except Exception as exc:
        response = await self.debugger.response_for(exc, request, scope)
    await response(scope, receive, send)
```

Key implementation choices

- No mandatory runtime dependency in the core package; Uvicorn and Jinja2 are optional extras.
- Route parameters are compiled into regular expressions and converted by registered converter functions.
- Endpoint signatures are inspected only to resolve path parameters, Request, WebSocket, and Schema types.
- Responses write standard ASGI start/body messages and use a custom JSON encoder for dates and dataclasses.
- Jinax imports Jinja2 lazily so API-only projects remain unaffected.
- The testing client invokes the ASGI application directly without starting a network server.

Prototype status

Version 0.1.0 Alpha: workable and tested, but not yet a stable, audited, internet-scale framework release.

24. Example Applications and Usage

Four examples demonstrate that the backend does not control the frontend.

Example	What it demonstrates
hello_api	JSON routes, typed parameters, schema injection, CORS, rate limiting, route groups, WebSocket echo.
jinax_site	Optional server-rendered HTML, inheritance, loops, autoescaping, async render response.
react_backend	CORS-enabled product API for a Vite/React development origin.
android_backend	Versioned device-registration endpoint validated for Android clients.

```
# Install from the extracted prototype folder
python -m venv .venv
# Windows PowerShell
.\venv\Scripts\Activate.ps1
python -m pip install -e ".[standard,dev]"

# Start the main example
flaxon run examples.hello_api.app:app --reload

# Inspect it
flaxon routes examples.hello_api.app:app
flaxon doctor examples.hello_api.app:app

# Test it
pytest
```

Example requests

```
GET http://127.0.0.1:8000/health
GET http://127.0.0.1:8000/api/users/7
POST http://127.0.0.1:8000/api/users
Content-Type: application/json

{"name":"Aldane","email":"aldane@example.com","age":20}
```

The React example only returns JSON. The Android example uses the same framework but a mobile-oriented schema. The Jinax example is isolated in its own directory and imports `flaxon.jinax` explicitly. This demonstrates that template support is a choice rather than a requirement.

25. Testing and Verification Results

Testing is part of the framework design because user applications depend on predictable routing, responses, validation, and errors. The prototype includes a synchronous `TestClient` and asynchronous `AsyncTestClient` that construct ASGI scopes and capture response messages without opening a TCP port.

Test area	Verified behavior
Core JSON route	200 response, correct JSON, request ID header.
Typed routing	Integer conversion and reverse URL generation.
Router prefix	Nested API path registration.
Validation success	Schema object injected with valid values.
Validation failure	422, FX-VAL-001, field messages.
Missing route	404 and FX-HTTP-404.
Debug failure	500 and structured development traceback.
Jinax	Async template render returns expected HTML.

Automated result

8 tests passed with no warnings after cleanup.

A network smoke test started Uvicorn on a local port, called GET /health, and posted a validated JSON user. The server returned 200 responses and completed startup/shutdown lifecycle events. CLI verification listed five HTTP routes and one WebSocket route. Doctor imported the application and produced one expected warning because the example intentionally lacks a production secret.

Required pre-1.0 testing expansion

- ASGI protocol conformance and disconnect edge cases.
- Streaming, large bodies, multipart uploads, cookies, and header duplication.
- WebSocket handshake, disconnect, room cleanup, backpressure, and distributed broadcast.
- Middleware order, exception propagation, cancellation, and graceful shutdown.
- Security tests for injection, path traversal, host attacks, CORS, CSRF where applicable, and secret leakage.
- Load, soak, memory, latency percentile, and multi-worker tests on supported Python versions.

26. Performance and Scalability Plan

Flaxon's speed strategy is architectural and measurable. ASGI permits concurrent network operations, but performance depends on the server, event loop, route matching, serialization, middleware, database client, application logic, and deployment. The framework should optimize overhead without sacrificing readable code or correctness.

Area	Optimization direction	Measurement
Routing	Precompile patterns; later use a radix/trie router.	Requests/second and route-match latency.
JSON	Compact encoding; optional faster serializer plugin.	Payload size and serialization time.
Middleware	Avoid unnecessary wrappers and repeated parsing.	Per-layer microseconds.
Validation	Cache schema metadata; efficient coercion.	Validation latency by payload size.
Templates	Cache compiled templates; disable reload in production.	Render latency and cache hit rate.
WebSockets	Bound queues; efficient room lookup; external broadcaster.	Connections, messages/sec, memory/connection.
Database	Pool connections; avoid N+1 queries; async clients.	Query latency and pool saturation.

Scaling model

1. Keep HTTP workers stateless and run more than one process when CPU and reliability require it.
2. Place durable data in databases and files in object storage.
3. Use Redis or another shared system for caching, rate limits, sessions, job queues, and WebSocket fan-out.
4. Move CPU-heavy jobs to workers or specialized services.
5. Use a reverse proxy or managed ingress for TLS, limits, health checks, and rolling deployment.
6. Profile before optimizing; publish reproducible benchmark configuration and percentiles rather than one headline number.

Realistic goal

Flaxon should make Python suitable for many chat, social, dashboard, and mobile backends, while allowing high-load subsystems to use optimized services or other languages when evidence supports that choice.

27. Security, Privacy, and Reliability

Security must be integrated into framework defaults and documentation. The prototype establishes autoescaped templates, structured validation, security headers, request identifiers, redaction, safe production errors, and basic rate limiting. These controls are foundations rather than a complete security program.

Threat	Framework/application control
Cross-site scripting	Jinja2 autoescape, output context awareness, Content Security Policy.
Injection	Parameterized database queries, schema validation, allowlists.
Credential theft	Secure password hashing plugin, MFA/OAuth options, secret storage.
Token abuse	Short access tokens, refresh rotation, revocation, audience/issuer checks.
Cross-site request forgery	CSRF tokens for cookie-authenticated state changes.
Denial of service	Body/time limits, rate limits, queue bounds, proxy protection.
Data leakage	Redaction, production-safe errors, least-privilege logs, retention controls.
Supply chain	Pinned/locked releases, dependency scanning, signed release workflow.
Broken authorization	Central policy checks, default deny, ownership tests.
Upload attacks	Type/size validation, isolated storage, malware scanning, random names.

Reliability requirements include graceful shutdown, timeouts, retries with idempotency, circuit breakers for external services, health/readiness endpoints, database transaction boundaries, backups, restore testing, and observable error budgets.

Retries must not duplicate payments, messages, or user actions.

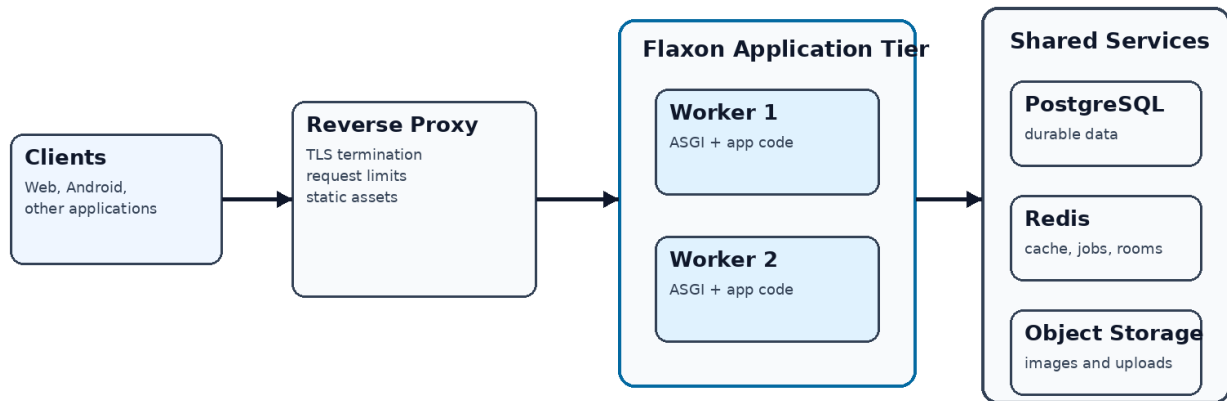
Privacy requirements include data minimization, purpose limitation, access logging, deletion workflows, encryption in transit and at rest where appropriate, regional/legal review, and careful treatment of debug snapshots. The framework should avoid collecting personal data by default.

Production rule

Debug mode, template auto-reload, development error pages, and example secrets must never be enabled in public production deployments.

28. Deployment and DevOps

Suggested Production Deployment Topology



Run multiple stateless workers. Store shared state in external services and use a distributed broadcaster for cross-worker WebSockets.

The prototype includes a non-root Dockerfile, Docker Compose file, and GitHub Actions test matrix. It runs Flaxon through Uvicorn. Uvicorn's deployment guidance explains that production configuration depends on the environment and supports worker-based deployment patterns [6]. Modern container guidance also recommends running as a non-root user [7].

```
docker build -t flaxon-demo .
docker run --rm -p 8000:8000 -e FLAXON_ENV=production -e FLAXON_DEBUG=false -e FLAXON_SECRET_KEY="replace-with-a-secret" flaxon-demo
```

A production pipeline should run linting, type checks, tests, security scans, package build, container scanning, staging deployment, smoke tests, and controlled promotion. Deployments should be reversible and database migrations should be reviewed for locking and backward compatibility.

- Expose /health for liveness and a dependency-aware readiness endpoint.
- Terminate TLS at a trusted proxy or managed platform and configure forwarded headers carefully.
- Use separate configuration and credentials for development, staging, and production.
- Send logs to centralized storage and correlate them with request IDs.
- Scale workers based on latency, CPU, memory, connection count, and downstream saturation.

29. Roadmap, Feasibility, Risks, and Conclusion

Milestone	Primary output
0.1 Core prototype	Current ASGI app, routing, HTTP, validation, WebSocket, Jinax, debugger, tests, CLI.
0.2 Protocol hardening	ASGI conformance, multipart, disconnects, streaming, middleware recovery.
0.3 Production services	Sessions, trusted hosts, timeouts, logging, health, OpenAPI.
0.4 Plugins	SQLAlchemy, Redis, auth, storage, observability interfaces.
0.5 Distributed systems	Queues, schedules, Redis broadcast, backpressure, deployment guides.
0.9 Release candidate	API freeze, security audit, benchmarks, migration guide, broad CI.
1.0 Stable	Documented compatibility, signed releases, governance, support policy.

Feasibility: the prototype proves that the central concept is technically achievable with standard ASGI messages and a small Python codebase. The greatest work is not the initial route decorator; it is long-term compatibility, protocol correctness, security, documentation, ecosystem development, and maintenance. An open-source community and narrow core scope improve feasibility.

Major risks and responses

- Scope explosion: keep advanced integrations optional and milestone-driven.
- Performance marketing without evidence: publish reproducible benchmarks and limitations.
- Security vulnerabilities: use established dependencies where appropriate, audits, disclosure policy, and fast patching.
- Maintainer overload: document governance, automate releases, and recruit contributors.
- Name/package conflicts: verify trademarks, domains, and package availability before public launch.
- API instability: mark alpha releases clearly and publish deprecation rules before 1.0.

Conclusion: Flaxon has a clear problem, differentiating principles, a practical architecture, and a working starting point. Its best path is not to recreate every framework feature immediately. It should become excellent at a small set of promises: approachable Python, async networking, technology freedom, readable failures, optional structure, and honest production engineering.

30. References and Execution Checklist

Selected primary technical references

Ref.	Source
[1]	ASGI Documentation. "HTTP & WebSocket ASGI Message Format." asgi.readthedocs.io/en/latest/specs/www.html
[2]	ASGI Documentation. "Lifespan Protocol." asgi.readthedocs.io/en/latest/specs/lifespan.html
[3]	Pallets. "Jinja Introduction." jinja.palletsprojects.com/en/stable/intro/
[4]	Pallets. "Jinja Template Designer Documentation." jinja.palletsprojects.com/en/stable/templates/
[5]	Python Packaging Authority. "Writing your pyproject.toml." packaging.python.org/en/latest/guides/writing-pyproject-toml/
[6]	Uvicorn. "Deployment." uvicorn.org/deployment/
[7]	Uvicorn. "Docker." uvicorn.org/deployment/docker/

Prototype execution checklist

1. Extract Flaxon_Production_Prototype.zip into a working directory.
2. Open PowerShell or a terminal in the extracted flaxon_prototype folder.
3. Create and activate a Python 3.11+ virtual environment.
4. Run `python -m pip install -e ".[standard,dev]"`.
5. Run `pytest` and confirm the test suite passes.
6. Run `flaxon routes examples.hello_api.app:app`.
7. Run `flaxon doctor examples.hello_api.app:app` and address production warnings.
8. Start with `flaxon run examples.hello_api.app:app --reload`.
9. Open `/health`, test the user endpoint, then study `src/flaxon/application/app.py`.
10. Create a feature branch before expanding database, auth, OpenAPI, or distributed WebSocket support.

Recommended first coding task

Add a trusted-host middleware and tests, then add a formal plugin interface. These features strengthen production safety and extensibility without changing the framework's central design.

End of document.